ARQUITETURA DE COMPUTADORES

Sandra Rovena Frigeri



sagah

Modelos de programação

Objetivos de aprendizagem

Ao final deste texto, você deve apresentar os seguintes aprendizados:

- Descrever os diferentes tipos de operações existentes na maioria dos processadores.
- Reconhecer a arquitetura baseada em pilhas e a arquitetura baseada em registradores.
- Escrever programas em linguagem de montagem para arquiteturas baseadas em pilhas e em registradores de propósito geral.

Introdução

O cérebro do computador é a unidade central de processamento (CPU, do inglês *central processing unit*). Ela é responsável pelo controle de toda a máquina, pela execução dos programas, pelo gerenciamento da entrada e saída de dados e pela transformação dos dados. A CPU é um *chip*, ou seja, um circuito integrado formado por diversos transistores.

Os programas que o computador executa são formados por instruções de máquina, que são armazenadas sequencialmente em células de memória e são executadas pela CPU. Cada instrução é um código binário, que é interpretado pela CPU e, então, executado. O conjunto de instruções que podem ser executadas é denominado linguagem de máquina, ou ISA (do inglês, *instruction set architecture*), e define a interface de programação entre o *software* e o processador. Consiste, portanto, na coleção completa de instruções de um computador.

Atualmente, a maioria dos programadores não constrói seus programas em linguagem de máquina. Eles utilizam linguagens de alto nível, que são compiladas ou interpretadas e, então, traduzidas para a linguagem de máquina. Assim, o conhecimento da linguagem de máquina não é essencial para a construção de sistemas, mas contribui para entender como as instruções serão executadas e obter um melhor aproveitamento dos recursos computacionais.

Neste capítulo, você vai estudar as instruções mais comuns dos computadores, verificando como elas são executadas e identificando suas características conforme o tipo de arquitetura de instruções, que pode ser baseada em pilha ou registradores, entre outras. Por fim, você vai verificar como construir programas em linguagem de máquina.

Instruction set architecture — ISA

O conjunto de instruções que pode ser executado pela CPU é denominado ***instruction set architecture***, ou ISA. Cada instrução é uma operação básica que pode ser realizada pelo processador. As instruções são utilizadas para construir programas, que são armazenados na memória principal e executados pelo processador. Os principais tipos de operações que podem ser executadas são:

- operações de movimentação processador–memória — especificam transferências de dados entre processador e memória;
- operações de movimentação processador–entrada/saída (E/S) — especificam transferências de dados entre dispositivos periféricos e o processador;
- operações de processamento de dados — especificam operações de transformações de dados realizadas pela unidade lógica e aritmética (ULA); e
- operações de controle — especificam operações de desvios nos programas.

Os processadores podem ser classificados como **RISC** ou **CISC** quanto ao conjunto de instruções e conforme sua estratégia de implementação. Os **processadores RISC** (do inglês *reduced instruction set computer*, ou computador com conjunto reduzido de instruções) possuem um pequeno conjunto de instruções simples, em que cada uma possui um ciclo de processamento muito rápido. Porém, é necessário combinar várias instruções para realizar operações mais complexas. Já os **processadores CISC** (do inglês *complex instruction set computer*, ou computador com conjunto complexo de instruções) possuem um grande conjunto de instruções complexas, que possuem um ciclo de processamento mais lento, mas já realizam diretamente as instruções mais complexas.

Processadores CISC são mais antigos e tinham a proposta de ter muitas instruções e ser mais compatíveis em nível de sistema. Os computadores atuais

possuem processadores RISC, que são muito rápidos e executam operações simples muito rapidamente; mesmo que precisem realizar um conjunto de operações para realizar operações mais complexas, ainda são mais rápidos do que os computadores CISC.

Vejamos o caso da expressão aritmética C = A + B, onde A e B são endereços de posições de memória que contêm os dados que devem ser utilizados para realizar a soma — eles formam os operandos; já C é o endereço da posição de memória onde deve ser armazenado o resultado. Qual é a diferença entre implementar essa expressão em um processador RISC ou em um processador CISC?

Considere algumas instruções básicas que um processador RISC pode ter (dentro de seu conjunto de instruções):

- LOAD, para ler um dado e movê-lo para um registrador;
- STOR, para guardar o resultado da ULA;
- ADD, para ativar a operação de soma na ULA.

Combinando essas operações, é possível especificar a sequência de instruções para executar a expressão C = A + B. Assim, de forma simplificada, podemos ter: LOAD A, LOAD B, ADD, STOR C. Em cada uma dessas instruções, tem-se um endereço, que corresponde à posição de memória que contém o dado a ser lido ou armazenado.

Por outro lado, nos processadores CISC, uma única instrução ADD A, B, C resolve a expressão aritmética. Nela estão indicados os operandos (A e B), a operação a ser realizada e onde deve ser guardado o resultado (C). É uma instrução com três endereços: A, B e C.



Fique atento

Perceba que, quanto menos endereços existirem em uma instrução, mais primitiva ela se torna, exigindo um processador menos complexo, pois a unidade de controle será mais simples. Por outro lado, quantos mais endereços a instrução possuir, mais complexa ela se torna e exige uma unidade de controle que seja capaz de decodificar todos os seus elementos.

Considere que você deseja resolver a seguinte expressão aritmética:

$$Y = \frac{A - B}{C + (DxE)}$$

Na Figura 1 você pode visualizar três alternativas de implementação para instruções com um, dois ou três endereços. Perceba que, quanto menos endereços a estrutura da instrução possui, mais instruções são necessárias para resolver a operação. Por outro lado, quando mais endereços a instrução possui, menos instruções são necessárias. Assim, na alternativa (a), que utiliza instruções com três endereços, são necessárias apenas quatro instruções. Já na alternativa (b), que utiliza instruções com dois endereços, são necessárias seis instruções. Por fim, na alternativa (c), que utiliza instruções com um endereço, são necessárias oito instruções. Nessa última alternativa, vê-se o uso de uma arquitetura da ULA que usa o acumulador (AC) como um de seus operandos. Nessa figura, você vê algumas instruções comuns, que são:

- SUB (subtração);
- MPY (multiplicação);
- ADD (adição);
- DIV (divisão);
- MOVE (atribuição);
- LOAD (carga/leitura);
- STOR (armazenar).

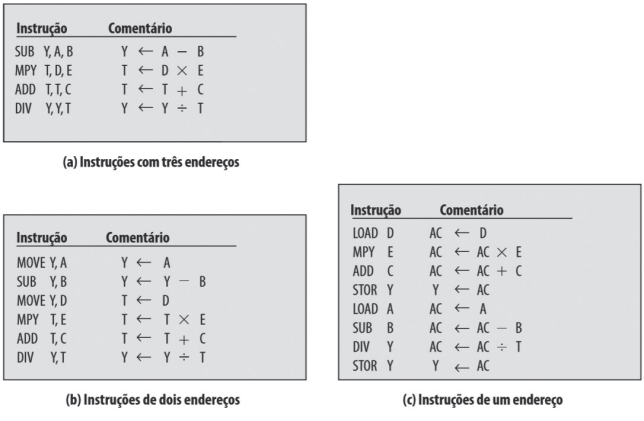


Figura 1. Comparação de programas com instruções com um, dois e três endereços.

Fonte: Stallings (2010, p. 209).

Instruções

Uma instrução é constituída por duas informações:

1. **Código da operação (*OpCode*)**: identifica a ação a ser realizada. Cada código binário especifica uma única instrução e será a entrada para o decodificador da unidade de controle, que vai gerar sinais de controle para sua execução.
2. **Código do operando**: especifica parâmetros para a operação. É o campo da instrução que possui o código binário da localização do dado (ou dados) que será processado. Ele pode se referir ao operando fonte, destino ou próxima instrução.



Fique atento

A quantidade de *bytes* do código de operação (*OpCode*) de uma instrução depende da quantidade de operações que a CPU pode realizar e do tipo de operação e dados a serem manipulados. Mas, de forma geral, os OpCodes possuem de 1 a 4 *bytes*.

O operando define os dados para a instrução, mas há instruções que não possuem operando, ou este pode estar implícito.

Quais são as possibilidades de operandos das instruções?

- Dado necessário para a realização da operação.
- Onde o resultado da operação será armazenado.
- Registrador, que poderá ter o dado ou endereço de um dado que será utilizado na operação.

Onde podem estar os dados que serão utilizados nas instruções, representados pelos operandos?

- **Memória principal ou virtual**: nesse caso, é necessário fornecer seu endereço.
- **Registrador do processador**: possuem identificadores próprios e, muitas vezes, são o padrão usado em algumas instruções; nesse caso, estarão implícitos.

- **Imediato**: quando o valor do operando está na instrução.
- **Dispositivo de E/S**: nesse caso, é necessário especificar o dispositivo ou endereço do dado.

O processador repete "infinitamente" o ciclo da instrução, como se pode observar detalhadamente na Figura 2. Nessa figura, a instrução é buscada, depois decodificada; então, buscam-se os operandos, a partir do cálculo do endereço; na sequência, é realizada a operação e, finalmente, calcula-se o endereço do destino onde será guardado o resultado; por fim, este é armazenado. Depois disso, começa o processo da próxima instrução, a partir do cálculo do endereço da instrução, e, então, reinicia-se o ciclo da instrução.

Conforme você viu anteriormente, em um processador RISC, esse processo pode ser mais simples, pois, de forma geral, cada instrução terá no máximo um endereço. Assim, o ciclo da instrução será: buscar a instrução; decodificar; buscar o operando (se necessário); realizar a operação; buscar a próxima instrução.

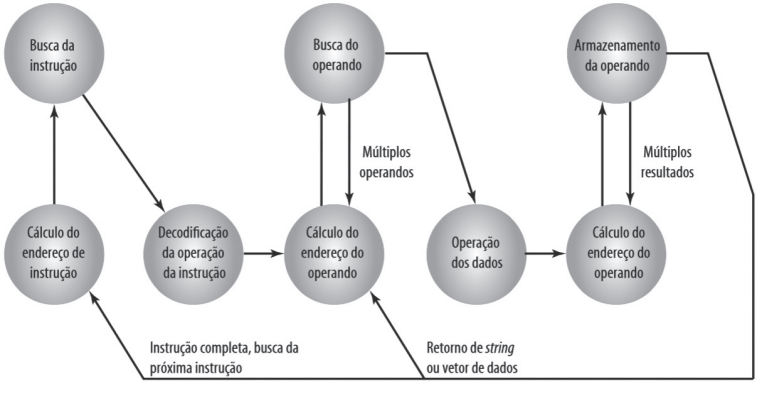


Figura 2. Diagrama de estados do ciclo da instrução.
Fonte: Stallings (2010, p. 59).

Identifica-se cada instrução pelo seu código de operação (*OpCode*), que é um código binário, por exemplo: 10010001. Porém, é complicado ler códigos binários. Assim, foram criadas representações simbólicas, chamadas **mnemônicas** (comandos mnemônicos), que são abreviaturas do termo em inglês que indica a operação. Esses comandos podem ser agrupados conforme o tipo de operação que executam. Assim, tem-se operações de transferência de

dados, aritmética, lógica, transferência de controle, entrada/saída e conversão. Lembre-se de que o conjunto de instruções e sua sintaxe vai depender do processador a ser utilizado.

No Quadro 1, você pode ver as instruções mais comuns.

Quadro 1. Operações comuns do conjunto de instruções

Tipo	Nome da operação	Descrição
Transferência de dados	*Move* (transferência)	Transfere palavra ou bloco da origem ao destino.
	Store (armazenar)	Transfere palavra do processador para a memória.
	Load (carregar)	Transfere palavra da memória para o processador.
	Exchange	Troca o conteúdo da origem e do destino.
	Clear (*reset*)	Transfere palavra de 0s para o destino.
	Set	Transfere palavra de 1s para o destino.
	Push	Transfere palavra da origem para o topo da pilha.
Aritmética	*Pop*	Transfere palavra do topo da pilha para o destino.
	Add	Calcula a soma de dois operandos.
	Subtract	Calcula a diferença de dois operandos.
	Multiply	Calcula o produto de dois operandos.
	Divide	Calcula o quociente de dois operandos.
	Absolute	Substitui o operando pelo seu valor absoluto.
	Negate	Troca o sinal do operando.
	Increment	Soma 1 ao operando.
	Decrement	Subtrai 1 do operando.

(*Continua*)

(*Continuação*)

Quadro 1. Operações comuns do conjunto de instruções

Tipo	Nome da operação	Descrição
Lógica	*AND*	Realiza o AND lógico.
	OR	Realiza o OR lógico.
	NOT (complemento)	Realiza o NOT lógico.
	Exclusive-OR	Realiza o XOR lógico.
	Test	Testa condição especificada; define *flag*(s) com base no resultado.
	Compare	Faz comparação lógica ou aritmética de dois ou mais operandos; define *flag*(s) com base no resultado.
	Set control variables	Classe de instruções para definir controles para fins de proteção, tratamento de interrupção, controle de tempo. etc.
	Shift	Desloca o operando para a esquerda (direita), introduzindo constantes na extremidade.
	Rotate	Desloca ciclicamente o operando para a esquerda (direita), de uma extremidade à outra.
Transferência de controle	*Jump* (desvio)	Transferência incondicional; carrega PC com endereço especificado.
	Jump conditional	Testa condição especificada; ou carrega PC com endereço especificado; ou não faz nada, com base na condição.
	Jump to subroutine	Coloca informação do controle do programa atual em local conhecido; salta para endereço especificado.
	Return	Substitui conteúdo do PC por outro registrador de local conhecido.

(*Continua*)

(*Continuação*)

Quadro 1. Operações comuns do conjunto de instruções

Tipo	Nome da operação	Descrição
Transferência de controle	*Execute*	Busca operando do local especificado e executa como instrução; não modifica o PC.
	Skip	Incrementa o PC para saltar para a próxima instrução.
	Skip conditional	Testa condição especificada; ou salta ou não faz nada, com base na condição.
	Halt	Termina a execução do programa.
	Wait (*hold*)	Termina a execução do programa; testa condição especificada repetidamente; retoma a execução quando a condição for satisfeita.
	No operation	Nenhuma operação é realizada, mas a execução do programa continua.
Entrada/ saída	*Input* (leitura)	Transfere dados da porta de E/S ou dispositivo especificado para o destino (por exemplo, memória principal ou registrador do processador).
	Output (escrita)	Transfere dados da origem especificada para porta de E/S ou dispositivo.
	Start I/O	Transfere instruções para o processador de E/S para iniciar operação de E/S.
	Test I/O	Transfere informações de *status* do sistema de E/S para destino especificado.
Conversão	*Translate*	Traduz valores em uma seção da memória com base em uma tabela de correspondências.
	Convert	Converte o conteúdo de uma palavra de uma forma para outra.

Fonte: Adaptado de Stallings (2010).

Projeto do conjunto de instruções

Quando o processador é projetado, é feita a escolha da arquitetura do conjunto de instruções. Isso influenciará no modo como as instruções serão codificadas e em suas possibilidades de parâmetros. O projeto do conjunto de instruções pode ser classificado em:

- arquitetura de pilha;
- arquitetura baseada em acumulador;
- ISA registrador–registrador ou *load*/*store*;
- registrador–memória.

Na **arquitetura de pilha**, o processador controla um conjunto de registradores organizados como uma pilha, e esta é utilizada como fonte de dados para o trabalho da ULA. Assim, os dados são empilhados e usados a partir do topo da pilha; quando o topo é utilizado, sai da pilha, e o dado seguinte torna-se o topo. Nessa arquitetura, as instruções são bem simples, pois somente é necessário conhecer o topo da pilha, onde são realizadas todas as operações, tanto de inclusão como de uso dos dados. Nesse caso, a unidade de controle deve apenas decodificar a operação que será realizada.

Já na **arquitetura baseada em acumulador** há maior complexidade, pois um dos dados vem sempre do acumulador, mas o outro virá da memória e precisará ser colocado no registrador que será utilizado pela ULA. Assim, a unidade de controle precisa decodificar a instrução e a fonte do dado que será usado pela ULA junto com o dado que estiver no acumulador.

Outra arquitetura é a ***load*/*store***, também chamada de **ISA registrador–registrador**, que é mais complexa do que as anteriores. Nela, os dados dos registradores que contêm os operandos da ULA precisam ser carregados; assim, a unidade de controle precisa decodificar a instrução e buscar os dois operandos, que podem estar em registradores ou em endereços de memória.

Por fim, a arquitetura mais complexa é a **registrador–memória**, na qual as instruções da ULA são realizadas utilizando-se registradores, que podem ter endereços de operandos que estão na memória. Assim, a unidade de controle precisa decodificar a instrução e identificar os operandos, que podem ter nos registradores endereços que referenciam dados que estão na memória e que precisam ser buscados para realizar a operação da ULA.



Fique atento

Observe que as arquiteturas simples, como a de pilha, possuem instruções simples e, portanto, também terão uma unidade de controle simples. Por outro lado, possuem um conjunto limitado de operações, e isso faz com que operações complexas precisem de mais operações para serem realizadas. Por exemplo, para somar dois números, nessa arquitetura, será necessário: empilhar o primeiro operando; empilhar o segundo operando; executar a operação de soma; empilhar o resultado na pilha. Portanto, são necessárias quatro operações. Por outro lado, na arquitetura registrador–memória, seria necessária apenas uma operação, que indicaria o registrador do primeiro operando, o endereço de memória do segundo operando e o registrador onde será guardado o resultado. No entanto, essa arquitetura exige uma unidade de controle mais complexa.

A Figura 3 traz um esquema da arquitetura da ULA baseada em pilha. Nessa figura, observa-se a pilha de registradores, o ponteiro para o topo da pilha, que é chamado de *stack pointer*, as interconexões entre a pilha e os operandos da ULA e a saída da ULA.

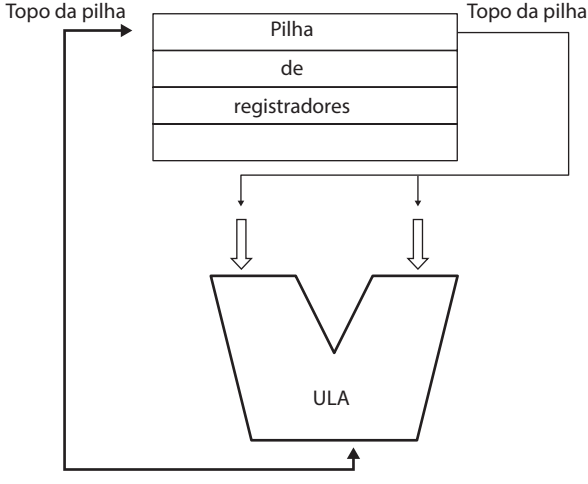


Figura 3. Arquitetura da ULA baseada em pilha.



Fique atento

Pilhas são estruturas que seguem o critério LIFO — *Last In First Out* —, ou seja, o último a entrar é o primeiro a sair. Nela, todas as operações são realizadas no topo da pilha. Assim, novos elementos são empilhados (*PUSH*) no topo da pilha; quando é necessário utilizar um dado, este é desempilhado também no topo da pilha, e essa operação se chama *POP*.

Linguagem de montagem

Como vimos, a linguagem de programação que um processador consegue decodificar é constituída por códigos binários e é denominada linguagem de máquina. Esta, para se tornar mais compreensível para o ser humano, é traduzida para mnemônicos; o conjunto destes constitui a linguagem de montagem. Assim, cada comando da linguagem de montagem é traduzido diretamente para uma instrução de máquina.

Programas escritos em linguagens de alto nível, como C, Java, PHP, entre outras, são inteligíveis para os programadores e são portáteis e independentes de arquitetura do processador, mas não são diretamente executáveis e, muitas vezes, não aproveitam adequadamente os recursos do *hardware*. Assim, aplicações que possuam restrições de desempenho críticas são, muitas vezes, escritas em linguagem de montagem. Porém, um programa em linguagem de montagem é muito mais difícil de ser escrito e pode ter de cinco a dez vezes mais instruções do que seu correspondente em alto nível.

O processo de montagem consiste na tradução do programa fonte — escrito em linguagem de montagem, chamada *Assembly* — para códigos binários da linguagem de máquina. Esse processo não é totalmente direto, pois, apesar de cada instrução da linguagem de montagem ter seu código correspondente em binário, é necessário resolver referências simbólicas dos endereços de desvios. Na Figura 4 você pode ver as etapas do processo de montagem. Ele tem início com a união dos programas fonte; a partir destes, são gerados os códigos-objeto, que, depois, são ligados (*linked*) com outros códigos-objeto, de bibliotecas, por exemplo; finalmente, é gerado o código executável. Este consiste no programa que é carregado para a memória principal para ser executado.

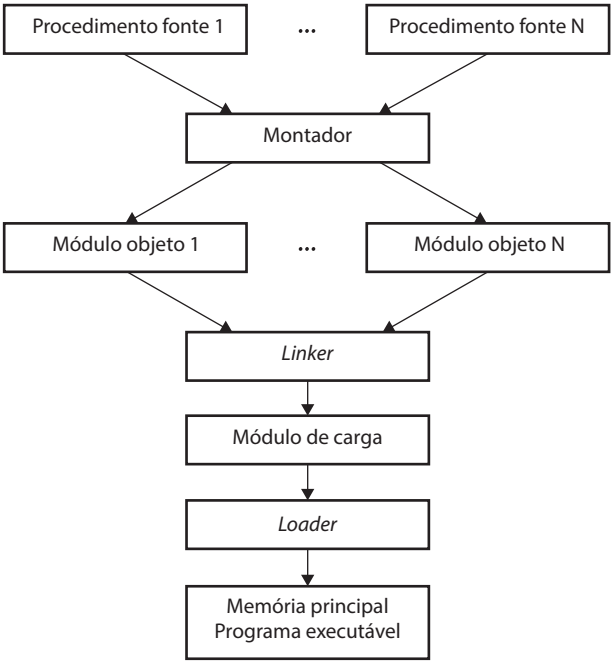


Figura 4. Etapas de montagem.

O uso atual da linguagem *Assembly* consiste na manipulação direta de *hardware* para sistemas que necessitam de performance crítica, pois permite a programação de *device drivers*, sistemas embarcados de baixo nível e sistemas de tempo real.



Fique atento

A linguagem Assembly é atrelada à arquitetura de uma determinada CPU; portanto, ela é completamente dependente do *hardware*. Cada família de processador possui sua própria linguagem Assembly; por consequência, programas escritos nessa linguagem não são portáveis.

Byte, *word* e *dword* são blocos de dados básicos do Assembly para que o processador possa trabalhar com o tamanho de dados adequado para executar

as instruções. Dessa forma, um *byte* possui 8 *bits*, um *word* possui 16 *bits* ou 2 *bytes*, e um *dword* possui 32 *bits* ou 4 *bytes*. Em *Assembly*, os números são na base hexadecimal, pois ficam menores, o que permite a fácil identificação dos *bits* ligados; cada dígito hexadecimal corresponde a quatro dígitos binários (valores entre 0 e 15).

Os dados manipulados pelas instruções do *Assembly* são tratados por meio dos registradores, normalmente identificados por AX, BX, CX e DX, além dos SI e DI (*source index* e *destination index*) e dos registradores de indexação da pilha (*stack*), SP e BP (*stack pointer* e *base pointer*). Outro registrador importante é o registrador de *flags*, que recebe sinais que ligam ou desligam *bits*, indicando o sucesso ou problemas em operações realizadas, como o estouro da pilha, a divisão por zero, entre outros.



Link

Você pode construir seus próprios programas em *Assembly* e testá-los. Comece com programas simples, que resolvam expressões aritméticas, por exemplo. Veja como fazer programas em *Assembly* acessando o *link* abaixo:

https://goo.gl/3RoF5M

Você também pode testar programas em *Assembly* utilizando um compilador *on-line*, disponível no *link* abaixo:

https://goo.gl/xXpzQJ



Referência

STALLINGS, W. *Arquitetura e organização de computadores*. 8. ed. São Paulo: Pearson Prentice Hall, 2010.

Leituras recomendadas

EXEMPLOS de programas em *Assembly*. [2018]. Disponível em: <https://www.docdroid.net/rvu3/exemplos-de-programas-assembly.pdf>. Acesso em: 12 dez. 2018.

LOBUR, J.; NULL, L. *Princípios básicos de arquitetura e organização de computadores*. 2. ed. Porto Alegre: Bookman, 2011.

SOUSA, J. P. *Primeiros passos na programação em Linguagem Assembly.* set. 2005. Disponível em: <https://paginas.fe.up.pt/~jmf/mp0506/dwnlds/mp1-0506-print.pdf>. Acesso em: 12 dez. 2018.

TANENBAUM, A. S. *Organização estruturada de computadores*. 6. ed. São Paulo: Pearson, 2013.

WEBER, R. F. *Fundamentos de arquitetura de computadores*. 4. ed. Porto Alegre: Bookman: 2012.

Conteúdo:

sagah